

Instructions. This homework involves writing an SQL script that creates and populates a set of tables. Submit the following to Canvas before midnight on the due date:

- (1) A single SQL script called **hw2.sql** containing all of your **CREATE TABLE**, **INSERT**, and constraint definitions. It must be possible for the grader to completely recreate your tables by running the script from within PostgreSQL. This means your script must run successfully more than once, where it drops existing tables before recreating them. Include your name and assignment number in SQL comments at the top of the file.
- (2) A single PDF file called **hw2.pdf** with screenshots showing that your script ran successfully, the contents of each table, and the error output from the failing inserts described below. Include your first and last name, the course (CPSC-321), and the assignment number at the top of the file.

Part A: Implement the Airline Database

Use SQL to implement the airline database you designed in Part A of HW-1. As you implement and populate your tables, you may need to revise your original design. Note any changes in your write-up.

For each table, choose appropriate data types and populate it with at least five rows. The rows you insert should exercise the various constraints on each table including primary, candidate, and foreign key constraints. Attributes should not accept NULL values except where necessary (e.g., optional attributes or foreign keys that may not apply).

In addition to primary keys and foreign keys, your implementation must enforce the following:

1. All route distances must be positive.
2. A route's departure and arrival airports must differ.
3. Booking prices must be nonnegative and stored with exactly two decimal places. Assume booking prices never exceed \$99,999.99.
4. A seat can be held by at most one passenger on a given flight and travel date. It can be the case that a seat assignment has not yet been determined for a booking.

Part B: Extend the Airline Database

Add the following to your airline database. For each table, choose appropriate data types and populate it with at least five rows. Define appropriate primary keys, foreign keys, defaults, and constraints to enforce the requirements below. Attributes are required unless stated otherwise.

1. Crew Members. Your database must also track crew members. Give each crew member a unique integer identifier generated automatically by PostgreSQL. Each crew member has a first name, a last name, a hire date, an optional end date (null if still employed), and a Boolean attribute indicating whether they are a pilot. Each crew member may have a supervisor, who is also a crew member. A crew member cannot supervise themselves. Assume the `is_pilot` attribute defaults to **FALSE** and that the end date, if present, must be on or after the hire date.

2. Crew Assignments. Your database must also track crew assignments. A crew assignment records that a crew member works a shift on a specific flight route and travel date, where each crew member can have at most one such assignment on a given flight route and travel date. Each assignment includes a starting date and time and an ending date and time. Shifts may cross midnight. The shift must end after it begins and cannot exceed 16 hours.

Part C: Failing Inserts

For this assignment, you must also include at least six **INSERT** statements that *fail* due to constraint violations. Demonstrate each of the following at least once:

- A duplicate primary key
- A foreign-key violation (referencing a nonexistent row)
- A NOT NULL violation
- A CHECK constraint violation
- A violation of a UNIQUE constraint other than the primary key

State the intended violation in a comment above each failing statement. Comment out each failing insert in your submitted script, but include the error output in your screenshots.